

Field Injection

Recommended:
Constructor injection

REST API

13. @RestController vs @Controller

@RestController
returns JSON directly

@Controller
returns view

14. Request parameter types

@RequestParam
query parameter

@PathVariable
URL path parameter

@RequestBody
JSON request body

15. Global Exception Handling

```
@ControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity handleException() {}

}
```

16. Validation

DTO

```
class UserDTO {
```

```
@NotNull
String name;

@email
String email;

}
```

Controller

```
public ResponseEntity create(@Valid UserDTO dto)
```

17. API Versioning

```
/api/v1/users
/api/v2/users
```

SPRING DATA JPA

18. Hibernate

ORM framework
Maps Java objects → database tables.

19. Repository Types

CrudRepository
Basic CRUD

JpaRepository
CRUD + pagination

PagingAndSortingRepository
pagination + sorting

20. Lazy vs Eager

Lazy
data loaded when accessed

Eager
data loaded immediately

21. N+1 Query Problem

1 query loads parent
N queries load children

22. Dirty Checking

Hibernate automatically detects changes in entities
and updates DB.

23. EntityManager

Core JPA interface to manage persistence.

TRANSACTIONS

24. @Transactional

Manages database transactions.

@Transactional
public void transfer() {}

25. Transaction Propagation

REQUIRED
REQUIRES_NEW
SUPPORTS

26. Isolation Levels

READ_COMMITTED
REPEATABLE_READ
SERIALIZABLE

MICROSERVICES

27. REST Client Types

RestTemplate
Blocking

WebClient
Reactive

FeignClient
Declarative REST client

28. API Gateway

Single entry point for microservices.

Responsibilities

- routing
- authentication
- rate limiting

29. Service Discovery

Services register with registry.

Example

Eureka

30. Circuit Breaker

Prevents cascading failures.

Example

Resilience4j

PERFORMANCE

31. API Optimization

- DB indexing
- caching
- pagination
- async calls

32. Caching

Redis

```
@Cacheable("products")  
public Product getProduct(Long id)
```

33. Large Dataset Handling

Pagination

PageRequest.of(page,size)

Streaming

Batch processing

SECURITY

34. Spring Security Filter Chain

```
Request  
|  
Security Filters  
|  
Authentication  
|  
Authorization
```

35. JWT Authentication Flow

Client login

|

Generate JWT

|

Client sends JWT in header

|

Server validates token

36. Authentication vs Authorization

Authentication

who you are

Authorization

what you can access

PRODUCTION

37. Spring Boot Actuator

Provides endpoints

/health

/metrics

/info

38. Monitoring Tools

Prometheus

Grafana

Datadog

39. External Config

Config Server

Kubernetes ConfigMap

Secrets

40. Debugging Production Issues

Logs
Metrics
Tracing
Rollback

SECTION 2 — 15 SYSTEM DESIGN SCENARIOS

1. URL Shortener

Client
|
API Service
|
Generate hash
|
Store mapping
|
DB + Redis cache

2. Notification System

API
|
Kafka
|
Worker Service
|
Email / SMS / Push

3. Rate Limiter

Use the Redis counter.

Key

userId:timestamp

4. File Upload System

Client
|
Presigned URL
|
Upload to S3
|
Store metadata

5. Ticket Booking System

Use

Optimistic Locking
or
Distributed Lock

6. Chat Application

Client
|
WebSocket Server
|
Message Broker
|
Database

7. Logging System

Application
|
Logstash
|
Elasticsearch
|
Kibana

8. Payment System

API
|
Payment Service
|
Bank Gateway

Use

Idempotency
Saga pattern

9. Audit Logging

Application

|

Kafka

|

Audit Service

|

Audit DB

10. Search System

Use

Elasticsearch

for full text search.

11. Distributed Cache

Redis Cluster

Features

TTL

Eviction

12. Job Scheduler

Quartz

Autosys

Cron Jobs

13. API Gateway Design

Gateway

|

Authentication

|

Routing

|

Rate limiting

14. Microservice Communication

REST

gRPC

Kafka

15. Multi Tenant Architecture

Shared database

Schema per tenant

Database per tenant

=====

SECTION 3 — REAL DEBUGGING STORIES

=====

Story 1 — Slow API

Situation

API response time increased to 4 seconds.

Investigation

Checked logs and DB queries.

Root Cause

Missing index.

Fix

```
CREATE INDEX idx_user_id  
ON orders(user_id)
```

Result

API response reduced to 150ms.

Story 2 — Memory Leak

Situation

Memory usage increasing continuously.

Root Cause

Large objects cached in static map.

Fix

Used Redis with TTL.

Result

Memory stabilized.

Story 3 — Connection Pool Exhausted

Error

HikariPool - Connection not available

Root Cause

Slow DB queries.

Fix

- optimized query
- increased pool size
- added proper transaction handling

Story 4 — Kafka Duplicate Processing

Situation

Orders processed twice.

Root Cause

Consumer retry.

Fix

Idempotent consumer.

Store processed eventId.

Story 5 — Circular Dependency

ServiceA → ServiceB

ServiceB → ServiceA

Fix

Refactored to

ServiceA

ServiceB

CommonService

Story 6 — Production 500 Errors

Situation

Random failures.

Investigation

Logs

Metrics

External API latency

Fix

Added

Circuit Breaker

Retry

Timeout

BEST WAY TO ANSWER DEBUGGING QUESTIONS

=====

- 1 Identify the issue
- 2 Check logs and metrics
- 3 Find root cause
- 4 Fix problem
- 5 Prevent future occurrence

=====

END OF NOTES

=====

maven

Maven Command	What It Does	What Happens Internally	Output
<code>mvn clean</code>	Deletes previous build files	Removes <code>target/</code> directory	Fresh build environment
<code>mvn validate</code>	Checks project structure	Validates <code>pom.xml</code> and project configuration	Ensures project is correct
<code>mvn compile</code>	Compiles source code	Converts <code>.java</code> → <code>.class</code>	<code>target/classes</code>
<code>mvn test</code>	Runs unit tests	Executes tests using JUnit / Surefire plugin	Test reports
<code>mvn package</code>	Packages application	Compiles + tests + creates artifact	<code>.jar</code> / <code>.war</code> in <code>target/</code>
<code>mvn verify</code>	Runs additional checks	Integration tests, quality checks	Verified build
<code>mvn install</code>	Installs artifact locally	Copies artifact to local Maven repo	<code>~/.m2/repository</code>
<code>mvn deploy</code>	Deploys artifact to remote repo	Uploads JAR to Nexus/Artifactory	Available for other projects
<code>mvn spring-boot:run</code>	Runs Spring Boot application	Starts embedded server	Application starts
<code>mvn dependency:tree</code>	Shows dependency hierarchy	Displays all project dependencies	Dependency graph
<code>mvn dependency:analyze</code>	Finds unused dependencies	Detects redundant libraries	Optimization info
<code>mvn sonar:sonar</code>	Runs SonarQube analysis	Checks bugs, code smells, coverage	Sonar dashboard report
<code>mvn clean install</code>	Clean + build + install	Deletes old build and installs new artifact	Local repository artifact
<code>mvn clean package</code>	Clean + build jar	Rebuilds project completely	Executable jar

microservice scenario

=====

how we implement rate limiting for particular user

=====

redis rate limiter config in properties

bucket 4j

=====

Micro vs mono arch:

=====

- 1) Architecture (as per business function they divide in module in micro)
- 2) tech stack (each module can have its own diff stack)
- 3) deployment (independently deploy)
- 4) use case (smaller app use mono, large complex flexible move toward micro)
- 5) scalability (vertical in mono ,horizontal in micro)

=====

key feature of microservice

=====

- 1) independent deployment
- 2) fault isolation
- 3) scalability

=====

component of microservices

=====

core char::

- 1) service independence
- 2) domain driven design (DDD)
- 3) API communication
- 4) Technology Diversity

service registry and discovery:

- 1) Api gateway
- 2) load balancer
- 3) data management
- 4) monitoring and logging
- 5) ci/cd pipeline

=====

API gateway

=====

- 1) centralise endpoint for managing client interaction. It simplifies communication by routing.
- 2) load balancing
- 3) authentication and authorization

=====

service discovery

=====

-> enable microservices to dynamically locate and communicate with each other within a distributed system

-> connect each other services without hard coded

=====

what is container

=====

lightweight, standalone unit that bundles an app along with its dependencies, including libraries, config and runtime

=====

how do microservice communicate with each other

=====

- 1) msg queue -> aws sqs , kafka , rabbitmq
- 2) grpc
- 3) synchronise -> rest

how would you handle microservice failure

=====

-> one failure lead to cascade failure

-> circuit breaker

-> retries and fallback

-> centralize logging

how can you ensure backward compatibility

=====

- 1) api versioning
- 2) semantic versioning principles

what is OAuth and why it is used ?

=====

what is cQRS pattern in microservices ? what prob it solve?

=====

compare api gateway and load balancer in microservices?

=====

What are some common design principles?

=====

- 1) high cohesion
- 2) autonomous
- 3) business domain centric
- 4) resilience
- 5) observable
- 6) automation

how do ensure security in microservices

=====

- 1) secure the communication
- 2) authentication and authorization
- 3) service level isolation

what are the few way to achive synchronous and asynchronous communication between microservice?

=====

=====

sync -> rest api calls,grpc,graphQL api

how do you implement rate limiting in microservices?

=====

- > token bucket algoritham
- > API gateway

What is an event driven arch in microservices?

=====

microservice communicate on event msgs

what is idempotency and why it is imp in microservices

=====

-> multiple request -> same type of response

what is database sharding and why used it is used in microservices?

=====

-> large database are divided into smaller more manageable pieces called "shard".

how do you handle data consistency in microservices

=====

- 1) saga pattern
- 2) event sourcing
- 3) distributed transaction

what is canary deployment in microservices?

=====

how would you monitor the microservices?

=====

- 1) distribution tracing
- 2) metrics collection (prometheus)
- 3) visualization and alerting (graphana and kibana)

what are DDD principles ?

=====

- 1) bounded context
- 2) ubiquitous languages
- 3) aggregates

how do you design for scalability in microservices?

=====

- 1) horizontal scaling
- 2) database partitioning (sharding)
- 3) caching
- 4) load balancing
- 5) asynchronous processing
- 6) service mesh

what is chaos engineering and how is it applied in microservice ?

=====

how do you perform logging in a distributed microservices system?

=====

- 1) centralized logging
- 2) structured logging
- 3) trace ids
- 4) log levels
- 5) security

what is the purpose of retry pattern in microservices

=====

what are sidecars in microservices?

=====

what are the best practices to deploy microservices in prod ?

=====

- 1) containerisation
- 2) ci/cd
- 3) deployment strategies
- 4) monitoring and observability
- 5) service discovery
- 6) security

java classic

we have two employee class which has two object both have same id and name ,now we have one set,now we want only unique employee elements to be stored in set

```
Employee e1 = new Employee(1, "Tejas");  
Employee e2 = new Employee(1, "Tejas");
```

```
Set<Employee> set = new HashSet<>();  
set.add(e1);  
set.add(e2);
```

```
System.out.println(set.size());
```

```
class Employee {  
    private int id;  
    private String name;  
  
    public Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    // 1. Override equals  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (!(o instanceof Employee)) return false;  
  
        Employee e = (Employee) o;  
        return id == e.id &&  
            Objects.equals(name, e.name);  
    }  
  
    // 2. Override hashCode  
    @Override  
    public int hashCode() {  
        return Objects.hash(id, name);  
    }  
}
```

Used_in_project

- 1) Component used in project
- 2) Config classes
- 3) Service
- 4) Repo
- 5)

Prod Defect

In our application, we use a **pipeline-based design pattern** where business logic is executed through a sequence of processing steps. A key part of this pipeline is a **mapper layer**, where we maintain precomputed logic for calculating risk scores. These risk calculations can be triggered from multiple entry points depending on the flow (e.g., assessment, self-assessment, etc.).

Problem:

Recently, a defect was introduced when an additional validation was added to ensure that **pre-calculation of risk should only be executed in an “assessment” context**.

However, this change unintentionally blocked the risk pre-calculation when the flow originated from the **self-assessment module**. As a result:

- After submitting a self-assessment, expected risk calculations were not triggered.
- This caused incorrect or missing risk scores for certain business rules.

Root Cause:

The newly added context check was too restrictive and did not account for the **self-assessment flow**, even though it also required the same pre-calculation logic.

Solution:

To fix this:

- I updated the **service layer (adapter class)** where the pipeline is orchestrated.
- Introduced conditional checks to correctly identify when recalculation should happen.
- Explicitly invoked the **mapper’s pre-calculation logic** for the self-assessment flow by passing the appropriate context.
- Ensured that the logic remains reusable and does not violate the original design constraints.

Outcome:

- Risk calculation now correctly triggers for both **assessment and self-assessment flows**.
- No regression in existing assessment behavior.
- Maintained separation of concerns by handling context-specific logic at the adapter/service layer instead of modifying core mapper logic.

Key Takeaways (for interview emphasis):

- Careful handling of **context-driven logic in pipeline systems**.
- Importance of identifying **side effects of restrictive conditions**.
- Fix applied at the right abstraction layer to avoid breaking existing design.